Comparing two complete lists of marked groups

Human Compression project: L071 recovery

Working proof; distinct review pending

Let $G \trianglelefteq A \leq \mathrm{Sym}(\Omega)$, and suppose conjugation induces every automorphism of G. The kernel of this action is allowed to be nontrivial. Let $\mathcal{T}$ be a finite list containing one representative of every automorphism orbit of the desired ordered generating tuples of G. Its completeness is a premise supplied by the acquired enumeration theorem and its completed execution.

Store $\mathcal{T}$ in a rooted tree whose edges carry permutations. Each path from the root records a literal initial segment of a tuple; identical initial segments have the same node. A terminal node records the tuple's rank and its position in the enumeration. When the list comes from the acquired stabilizer recursion, distinct children at a node lie in distinct orbits of the pointwise stabilizer of the preceding tuple. The comparison below also verifies each final match explicitly.

Let H be a second concrete group, let $f : H \longrightarrow G$ be a verified isomorphism, and let $\mathcal{S}$ be a finite list of tuples in H. The two groups need not have the same permutation degree.

Theorem 1. *Suppose each $s \in \mathcal{S}$ is mapped, by f followed by a recorded conjugation in A, to a terminal tuple of $\mathcal{T}$. Suppose these terminal tuples are distinct, and $|\mathcal{S}| = |\mathcal{T}|$. Then $\mathcal{S}$ and $\mathcal{T}$ give precisely the same classes of ordered marked groups, each once. All properties of the tuples that are invariant under group isomorphism transfer between the two lists.*

Proof. Conjugation by an element of A restricts to an automorphism of G. Each recorded equality therefore gives an isomorphism from the source marked group to its named terminal tuple. Distinct terminal tuples represent distinct automorphism orbits by the acquired enumeration theorem. Thus the resulting assignment is injective on marked classes. The equality of the two finite list sizes makes it surjective. In particular it transfers generation, the orders of adjacent products, the intersection property and any stated equivalence with a reversed marking. The assertion concerns ordered isomorphism classes; no reversal is silently added to their equivalence relation. □

Here is an explicit way to find and certify the conjugations. Start with $t = f(s)$, $a = 1$ and $C = A$. At a node with children marked by r, find $h \in C$ such that $t_i^h = r$ for the next position i. Replace the whole tuple by t^h, replace a by ah, and replace C by $C_C(r)$. Move to the child marked r and continue. If no such child exists, the comparison fails; it does not discard the source tuple.

Inductively, C centralizes every already matched mark. Thus the new conjugation preserves the entire matched initial segment, and the updated centralizer is exactly its pointwise stabilizer after adjoining r. For right conjugation, the accumulated element is $a = h_1 \cdots h_i$. At the end the checker verifies both $a \in A$ and the complete literal equality $f(s)^a = t_{\mathrm{terminal}}$. It also checks that the named terminal node exists and has not occurred for an earlier source tuple. These checks are the hypotheses of the theorem; a successful orbit-search call alone is not substituted for the final equality.

The concrete source comparison. The auxiliary archive supplied by Hartley and Hulpke records permutation generators and lists of indices selecting each marking. Our reader accepts only the two empty-list initializations, literal permutation additions, and tuple additions using

already defined indices. It reconstructs data from these expressions and does not execute the source's loader program. The original archive and every parsed file are retained with their exact hashes. They are comparison inputs acquired after our enumeration; neither their counts nor their generators direct its search.

For a nonempty source list, take H to be generated by its first tuple. The execution checks $|H| = |G|$ and membership in H of every source generator, constructs f by an exact group-isomorphism algorithm, and checks that f is bijective. It records generators of H, their images, all source-generator images and the individual conjugations. Empty source lists are compared only with empty completed enumerations. Thus absence claims retain the same completeness premise as positive classifications.

Verification boundary and attribution. The finite group computations use the retained exact GAP algorithms for group isomorphism, membership, centralizers and conjugating elements. They are computational premises, not Lean declarations. This comparison does not establish the completeness of $\mathcal{T}$ from agreement of totals; its complete enumeration proof remains a separate paid premise. The source markings and their classification are attributed to Hartley and Hulpke, and the subsequent verification to Leemans and Mixer. The present contribution is a recorded bijection from an independent enumeration to those original markings, using the same stabilizer structure as the common search theorem. No new classification priority is claimed.